

The Living Map: Mine/Tunnel Rescue

IEEE TSYP14 - AESS x RAS Technical Challenge - Phase 1 Technical Report

1. Overview

In disaster response, robots exploring collapsed structures, mines or contaminated zones often operate with no GPS and no communication network. If a robot fails mid-mission, the knowledge it gathered is lost with it. The Living Map gives the unmapped space its own memory: a **Writer** robot explores autonomously and writes what it finds as compact radio beacons; an **Outside Network Area** receives them, translates robot coordinates to real GPS and forwards them to a **Command Post**; the Command Post briefs an **Executor** robot, which enters and continues the mission from the inherited beacon information instead of starting from zero. No robot ever talks to the Command Post directly.

Environment: Mine/Tunnel Rescue, modelled on a Gafsa-style room-and-pillar mine (branching galleries, a dead-end chamber, a rock-fall-blocked branch). **Events detected:** toxic gas, structural collapse, trapped worker.

What is new in this design. (1) Beacon lifetime and rebroadcast rate are decoupled and both driven by priority, so urgent findings live longest and are re-sent most often; during development we found that counting one TTL unit per rebroadcast would have given urgent beacons the *shortest* life ($20 \times 2 \text{ s} = 40 \text{ s}$) and fixed it. (2) Beacon positions come from SLAM, not raw odometry (0.14 m mean error versus 4-8 m). (3) The hand-over survives a bad radio: ACK, retry and de-duplication give the same result with 50 % of sends failing. (4) Writer and Executor share one marked entrance frame, so the Executor's navigation does not depend on the GPS calibration. (5) The same firmware code runs on the ESP32 and in a host simulation, which lets us test the real logic (119 automated checks) before any hardware exists.

2. Technical Solution and Architecture

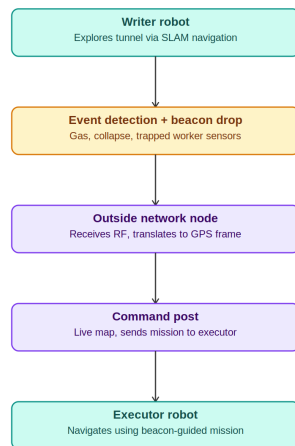


Figure 1 - Living Map system architecture (Mine/Tunnel Rescue)

Fig. 1 - System chain: Writer, beacon, outside network, command post, Executor.

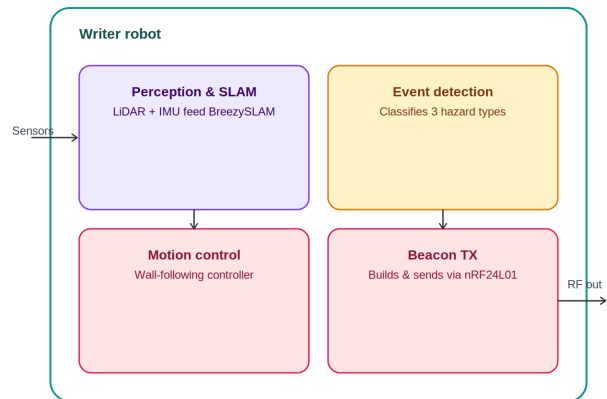
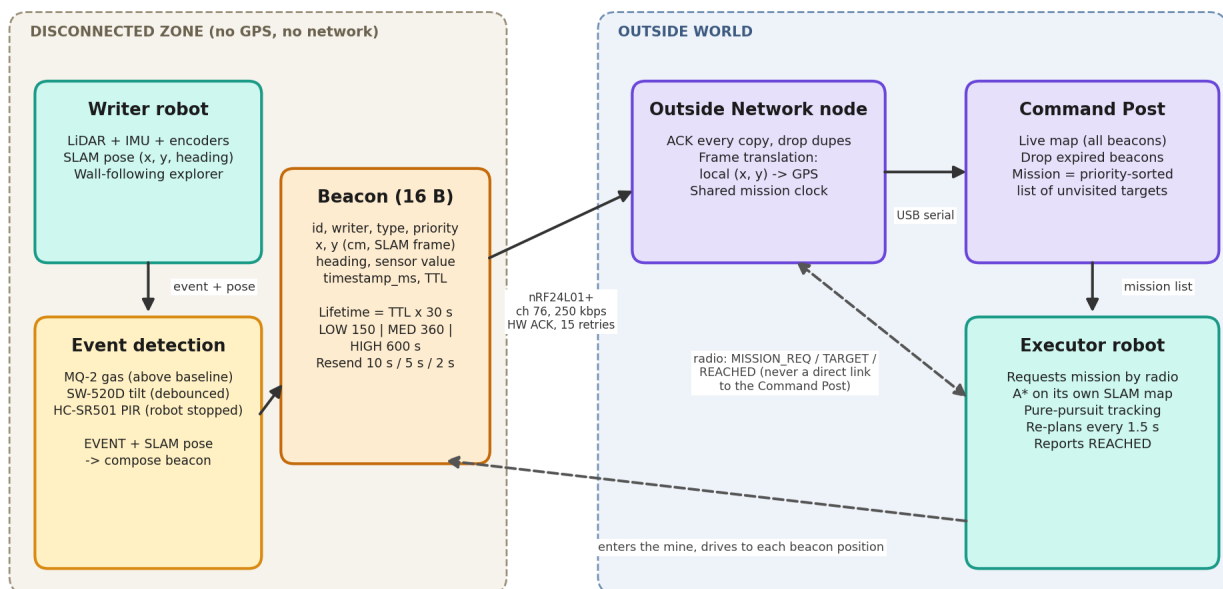


Figure 2 - Writer robot internal subsystems.

Fig. 2 - Writer robot internal subsystems.

Living Map: data flow and message formats



Data flow: sensor -> EVENT -> BEACON_TX -> radio -> Outside node (ACK, GPS translation) -> BEACON_RX -> Command Post -> TARGET list -> Executor

Fig. 3 - Data flow: what travels on each link, and where translation, ACK, aging and mission building happen.

Writer robot autonomy

A simulated 2D LiDAR (360 rays, 8 m range, PyBullet ray casting) and a noisy IMU/odometry model feed BreezySLAM (RMHC scan matching) to build an occupancy map and estimate pose without GPS. Exploration is a right-hand-rule wall follower with four states (FOLLOW, TURN_LEFT, SEEK, RECOVER), using the scan and the SLAM map for stuck detection and look-ahead. Stock scan matching drifts in heading along long featureless corridors (about -0.2 deg/m); we mitigate it with damped odometry/scan-match fusion and a wall-direction heading reference, which assumes an orthogonal layout (see Failure Cases).

Event detection and beacon deposition

Each event is stamped with the SLAM-estimated position, which goes into the beacon, and the ground-truth position, which is kept only to measure error. In the simulation an event fires when the robot enters a hazard zone; on hardware the detectors below produce the same EVENT. The host then composes the beacon from that event and the current SLAM pose, and the Writer keeps re-sending it until the outside node acknowledges it or its TTL runs out. The host drops repeats of the same incident within 3 m.

3. Beacon Message and Signal Design

A beacon is a packed 16-byte structure (comms/firmware/beacon_protocol.h, mirrored field for field in comms/beacon.py), well inside the 32-byte nRF24L01+ payload.

Offset	Bytes	Field	Meaning
0	1	beacon_id	unique id of this beacon
1	1	writer_id	which Writer dropped it
2	1	event_type	0 explored, 1 gas, 2 collapse, 3 trapped worker
3	1	priority	0 low, 1 medium, 2 high (derived from event type)
4	2	x_coord_cm	signed, SLAM frame, cm (range about +/-327 m)
6	2	y_coord_cm	signed, SLAM frame, cm; +y is to the left
8	1	heading	0-255 mapped to 0-360 deg
9	2	sensor_value	raw reading (gas level, vibration count, ...)
11	4	timestamp_ms	ms since mission start (shared mission clock)
15	1	ttl	initial lifetime in aging ticks

Radio link. nRF24L01+, 2476 MHz (channel 76), 250 kbps for best sensitivity, hardware ACK with 15 retries. The packet is type | source | destination | sequence | payload (at most 32 bytes); types are BEACON, ACK, HEARTBEAT, TARGET, REACHED and MISSION_REQ. Each node has a fixed address (LMNT plus one byte: hub 0x10, Writer 0x01, Executor 0x02), so both robots only ever talk to the outside node.

Message aging. One TTL unit is lost every 30 s of mission time since the beacon was written, so a beacon lives **150 s** (low, TTL 5), **360 s** (medium, TTL 12) or **600 s** (high, TTL 20). Priority also sets the rebroadcast interval (10 s, 5 s, 2 s). Expired beacons stay on the live map, marked expired, but are removed from the Executor's mission. All nodes assume one mission clock started at deployment.

4. Frame Translation

The Writer works in a private frame whose origin is the entrance pose, with +x along the entrance heading and +y to the left. A one-time manual calibration at the entrance records the GPS fix (lat0, lon0) and the compass bearing *theta* of local +x, clockwise from North. With x, y in metres (cm / 100), the outside node computes:

```
North = x cos(theta) + y sin(theta)
East  = x sin(theta) - y cos(theta)
lat   = lat0 + North / 111320
lon   = lon0 + East / (111320 cos(lat0))
```

Worked example (the values used in our tests): entrance 34.4250 N, 8.7842 E, theta = 90 deg (the tunnel heads east). A beacon at x = 1250 cm, y = -300 cm gives North = -3.00 m and East = +12.50 m, so the beacon is at **34.424973 N, 8.784336 E**, 12.5 m east and 3 m south of the entrance. This is the exact code path run by comms/outside_network.py and frame_translation.h; test_pipeline.py checks it.

Assumptions and error budget. The flat-earth approximation is appropriate at tunnel/mine scale. Manual calibration replaces an onboard magnetometer, which is unreliable near metal and ore. A heading error of 2 deg moves a beacon 100 m from the entrance by about 3.5 m (100 m x sin 2 deg). The Executor does not use GPS to navigate: it drives to the beacon's local coordinates in the shared entrance frame, so calibration error affects only the map shown at the Command Post.

5. Outside Network Area

The outside node (an ESP32 with an nRF24L01+) is the only gateway. It acknowledges every beacon copy, forwards only the first (de-duplication), applies the frame translation above and passes the beacon with its GPS position to the Command Post over USB serial. In the other direction it relays the mission list and receives the Executor's REACHED reports, which are repeated until acknowledged. The Command Post keeps the live map (all beacons, expired ones marked) and builds the mission as a priority-sorted list of unvisited targets. **Prototype scope:** the Command Post runs on a laptop at the mine entrance; the same frames can be carried over a cellular or satellite IP link to a distant post, but that long-haul link is designed, not built.

6. Executor Robot

Before entry the Executor asks the outside node for the mission and receives every target (it repeats the request until the whole list has arrived). It then uses the same navigation stack as the Writer (LiDAR + BreezySLAM) and plans with A* over its own occupancy map (walls inflated 0.35 m, unknown space passable but costlier), follows the path with pure pursuit, and re-plans every 1.5 s or when blocked. Targets are visited in priority order and each arrival is reported back through the outside node.

7. Event Detection Sensors

All three sensors use a plain analog or digital pin (no I2C/SPI) to keep wiring and debugging simple.

Event	Sensor	How it is used	Main limitation
Toxic gas	MQ-2	Analog output via a divider (ESP32 ADC is 3.3 V max); triggers when readings stay more than 350 ADC counts above a slowly learned clean-air baseline for 1.5 s; 60 s warm-up ignored.	Not selective; drifts with temperature and humidity, so it is an anomaly trigger, not a measurement.
Structural collapse	SW-520D tilt switch	Binary tilt/vibration; firmware debounces and needs repeat triggers in a short window.	Also trips on the robot's own bumps.
Trapped worker	HC-SR501 PIR	Detects a moving warm body up to about 7 m / 120 deg; warm-up of 30-60 s ignored.	Most reliable with the robot stopped; cannot see a motionless worker.

8. Results (simulation)

Test	Result
Writer (PyBullet mine, default seed)	448 s simulated, 146 m driven, returned to start, 0 wall contacts, 3 of 3 event zones reached
Writer position error vs ground truth	mean 0.14 m, max 0.28 m (odometry alone ends 4-8 m off)
Beacon position error	equals the SLAM error at the moment of the event (mean 0.14 m), because the beacon stores the SLAM pose
Executor	3 of 3 beacons in priority order, 136 s, 63 m, max error 0.28 m, 0 wall contacts; also 3 of 3 on seeds 1-3
Full system, real firmware in the loop	Writer mean error 0.25 m (max 0.45 m), 0 wall contacts; 3 beacons acknowledged and delivered; Executor 3 of 3 in 144 s, max error 0.26 m. Same result with 50 % of radio sends failing (duplicates discarded)
Automated checks	57 unit checks on the portable firmware core, 62 behaviour checks on the firmware running on a simulated bench, plus the comms pipeline tests

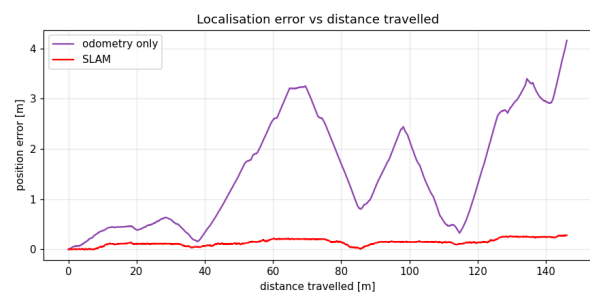


Fig. 4 - Position error: SLAM versus odometry only.

9. Implementation Plan (Phase 2 roadmap)

Item	Qty	Est. cost	Notes
ESP32 dev board	3	\$15-21	Writer, Executor, outside-network node
Motor driver, DC gear motors with encoders, chassis	2 sets	\$16-25	Differential drive per robot
MQ-2 / SW-520D / HC-SR501	1 each	\$5-9	Event sensors (Writer only)
nRF24L01+ radio	3	\$4-8	PA+LNA version if range tests demand it
Battery, regulation, wiring	-	\$13-20	5 V motor/sensor rail, 3.3 V logic
Base total		about \$55-75	Navigation sensing extra: ultrasonic+servo \$26-40, or 2D LiDAR \$146-210 (matches the simulation)

Build order: one robot driving; LiDAR/IMU and SLAM on hardware; event sensors tested one by one; radio link and beacon TX/RX; second robot with the goal-seeking controller; outside node; full integration in a mock tunnel with safe stand-ins for hazards.
Timeline: about 8 weeks (parts and robot 1 in weeks 1-2, SLAM and sensor bench tests 3-4, radio and range tests 5, robot 2 and outside node 6, integration 7, fixes and demo recording 8). Biggest risks: part shipping, SLAM tuning with real sensor noise, and radio range underground. Details: [implementation_plan.pdf](#).

10. Failure Cases

Failure	Why it happens	Mitigation / what remains
SLAM heading drift in featureless corridors	Scan matching has nothing to anchor to (about -0.2 deg/m; worse while turning).	Damped fusion, no correction while turning fast, wall-direction reference. Reference assumes orthogonal layouts; disable for irregular tunnels.
Sensor false positives and negatives	MQ-2 drifts and is not selective; PIR fires on warm moving air and misses still workers; tilt switch trips on bumps.	Baseline-rise trigger, debounce and repeat windows. Simulation does not exercise this; Phase 2 bench tests will.
Radio range underground	Rock and metal cut nRF24L01+ range; beacons are lost if the Writer is out of range.	Priority TTL/rebroadcast helps only while in range. Future: relay nodes, PA+LNA radios.
Frame-translation assumptions	Flat earth, one-time manual calibration; 2 deg error is about 3.5 m at 100 m.	Executor unaffected (uses the shared entrance frame). Re-calibrate if the entrance reference moves.
Executor navigation	Unknown space treated as passable, so it can enter unexplored dead ends before re-planning; gains reused from the Writer; static tunnel assumed.	An earlier straight-line controller reached 1 of 3 beacons; A* + pure pursuit reaches 3 of 3 on four seeds.
Simulation only; no long-haul uplink	Nothing here has run on the real ESP32 toolchain or hardware; wheel slip and sensor noise will be worse. The distant-command-post link is not built.	Phase 2 repeats every test physically and adds the uplink.

Details: [failure_cases.pdf](#), [robot_architecture.pdf](#), [sensor_research.pdf](#).